

Deadswitch

Onchain Inheritance & Dead Man's Switch Protocol on Solana

Version: 1.0.0 **Date:** April 13, 2026 **Authors:** Deadswitch Labs **Network:** Solana Mainnet-Beta **License:** MIT (Open Source)

Table of Contents

1. [Abstract](#)
 2. [The Problem](#)
 3. [Existing Solutions & Why They Fail](#)
 4. [Introducing Deadswitch](#)
 5. [How It Works — The Simple Version](#)
 6. [Technical Architecture](#)
 7. [Smart Contract Design](#)
 8. [Protocol Mechanics](#)
 9. [Security Model](#)
 10. [User Experience](#)
 11. [Business Model](#)
 12. [Market Analysis](#)
 13. [Roadmap](#)
 14. [Team](#)
 15. [References](#)
-

1. Abstract

Every year, billions of dollars in cryptocurrency become permanently inaccessible because their owners pass away, become incapacitated, or simply lose the ability to access their wallets. Unlike traditional bank accounts, there is no "next of kin" process for crypto. When a private key dies with its owner, the funds are gone forever.

Deadswitch is a non-custodial, permissionless protocol built on Solana that solves this problem. It monitors your wallet for activity. If you stop using your wallet for a period you define — say, 90 days — Deadswitch automatically distributes your crypto assets to the people you chose as beneficiaries. Your family, your partners, your causes. No lawyers. No probate courts. No seed phrases written on paper hidden in a drawer.

If you're still alive and active, your normal wallet usage (swapping tokens, sending SOL, minting an NFT — anything) automatically resets the timer. You don't need to do anything extra. Just keep using your wallet as you normally do.

Deadswitch never holds your funds. It never has access to your private keys. It simply watches, waits, and executes your wishes when the time comes.

2. The Problem

2.1 The Scale of Lost Crypto

Cryptocurrency was designed to give people full control over their money. But full control comes with a harsh reality: if you're the only one who can access your funds, what happens when you can't?

The numbers are staggering:

- **An estimated 3.7 million Bitcoin** (worth over \$170 billion at current prices) are believed to be permanently lost, many due to death or incapacitation of the holder. [1]
- **Chainalysis estimates** that approximately 20% of all Bitcoin in existence is held in wallets that haven't moved funds in over 5 years — many of these belong to deceased individuals. [2]
- As the first generation of crypto adopters ages, this problem will grow exponentially. The crypto inheritance crisis is not a future problem — it is happening right now.

2.2 Real Stories

These are not hypothetical scenarios. They happen every day:

- **Matthew Mellon**, a billionaire cryptocurrency investor, passed away in 2018 with an estimated \$500 million in XRP. His family struggled for years to recover the assets, and it is unclear how much was ever retrieved. [3]
- **Gerald Cotten**, the CEO of QuadrigaCX, died in 2018, reportedly taking the only passwords to \$190 million in customer funds to his grave. [4]
- **Countless everyday holders** — parents, siblings, friends — pass away without ever telling anyone how to access their crypto. No amount of legal paperwork helps if no one knows the seed phrase.

2.3 Why This Problem Is Unique to Crypto

In traditional finance, this problem doesn't exist at the same scale:

Traditional Finance	Cryptocurrency
Banks have "next of kin" processes	No centralized authority to contact
Courts can issue orders to transfer accounts	No court can override a blockchain
Lawyers and executors manage estate distribution	Private keys are the only access method
FDIC insurance protects deposits	Lost keys = lost funds, permanently
Government IDs can prove inheritance claims	Wallets are pseudonymous

The fundamental design of cryptocurrency — "be your own bank" — is also its greatest vulnerability when it comes to inheritance. Deadswitch bridges this gap without compromising the self-custody principles that make crypto powerful in the first place.

2.4 The "Grandma Problem"

Even when someone tries to plan ahead, the solutions are painful. Imagine telling your 70-year-old mother:

"Here's a piece of paper with 24 random words on it. When I die, download an app called Phantom, create a wallet by clicking 'Import,' type in these 24 words exactly, make sure you pick the right derivation path, and then you'll see my SOL. Oh, and don't let anyone else see this paper, and don't lose it, and don't type it into any website that looks even slightly suspicious."

This is not a viable inheritance plan. Deadswitch exists because your family should receive your crypto the same way they'd receive a bank transfer — directly into their wallet, automatically, without needing to become a crypto expert.

3. Existing Solutions & Why They Fail

3.1 Writing Down Your Seed Phrase

How it works: You write your 12 or 24-word recovery phrase on paper and store it somewhere safe (a safe, a safety deposit box, with a lawyer). You tell a trusted person where to find it.

Why it fails:

- If found by the wrong person, all your funds can be stolen instantly.
- The trusted person needs to know how to use the seed phrase — most non-crypto-native people don't.
- Paper degrades, gets lost in fires or floods, or simply gets thrown away.
- There is no way to split assets among multiple beneficiaries — whoever has the seed phrase gets everything.
- No time delay or verification — anyone who finds it can drain the wallet immediately.

Risk level: High. This is the most common "plan" and the most fragile.

3.2 Multisig Wallets

How it works: You create a wallet that requires multiple signatures to move funds (e.g., 2-of-3). You give one key to yourself, one to a family member, and one to a lawyer. If you die, the other two can cooperate to move the funds.

Why it fails:

- Requires all parties to be technically competent.
- Ongoing coordination burden — what if the lawyer retires, or the family member loses their key?
- Expensive to set up and maintain on most chains.
- Does not automate anything — your beneficiaries still need to know what to do, coordinate with each other, and execute transactions.
- Not designed for inheritance — it's a security tool being forced into a use case it wasn't built for.

Risk level: Medium. Better than a seed phrase on paper, but operationally complex.

3.3 Centralized Custody Services (e.g., Casa, Coinbase)

How it works: You store your crypto with a centralized service that has its own inheritance or account recovery processes.

Why it fails:

- Contradicts the core principle of self-custody — "not your keys, not your crypto."
- Subject to the service's terms, solvency, and regulatory environment. If the company goes bankrupt (see: FTX, Celsius, BlockFi), your inheritance plan goes with it.
- Often requires extensive legal documentation, identity verification, and months-long processes.
- Limited to assets and chains the service supports.

Risk level: Medium-high. You're trusting a company to outlive you and honor your wishes.

3.4 Sarcophagus Protocol

How it works: Sarcophagus is a decentralized dead man's switch built on Ethereum. You encrypt a file (like a seed phrase or a will), split the decryption key among a network of node operators called "archaeologists" using Shamir's Secret Sharing, and set a timer. If you fail to manually "re-wrap" (check in) before the timer expires, the archaeologists publish their key shards, allowing your designated recipient to decrypt the file.

Why it fails:

Problem	Impact
It reveals secrets, not assets	The recipient gets an encrypted file (usually a seed phrase). They still need to know how to import it and use it. The "grandma problem" is not solved.
Cold-start problem	The network needs archaeologist node operators to function. Too few users means too little revenue for operators, which means fewer operators, which means less reliability. As of 2025, the network is thin and struggling.

Manual check-in required	You must remember to "re-wrap" before the deadline. If you go on a long vacation and forget, your switch triggers prematurely — a false positive that could expose your secrets.
Requires a custom token (SARCO)	Archaeologists are paid in SARCO tokens. The token has low liquidity and adds friction. You need to acquire SARCO just to use the protocol.
Ethereum gas costs	Creating and managing a sarcophagus on Ethereum mainnet is expensive. Arbitrum support helps, but adds complexity.
Not on Solana	Zero presence in the Solana ecosystem.

Risk level: Medium. Technically sound concept, but poor UX and adoption challenges have limited its real-world impact.

3.5 The Gap Deadswitch Fills

None of the existing solutions achieve all of these at once:

- Automatically move actual assets (not just reveal secrets)
- Require zero technical knowledge from beneficiaries
- Work without manual check-ins
- Operate without a custom node network or token
- Stay non-custodial (your keys, your crypto, always)
- Be cheap enough for everyday users (not just whales)

Deadswitch does all six.

4. Introducing Deadswitch

4.1 What Deadswitch Is

Deadswitch is a **non-custodial inheritance protocol** built on Solana. It allows any crypto holder to create a "vault" — a smart contract that holds instructions for how their assets should be distributed if they become inactive.

The three core principles:

1. **Non-custodial.** Deadswitch never holds your private keys. Your assets stay in your wallet until the moment of redistribution. Even then, only the specific assets you designated are moved, and only to the addresses you chose.
2. **Passive proof-of-life.** You don't need to "check in" or press a button to prove you're alive. Any normal wallet activity — a swap, a transfer, a stake, an NFT mint — automatically resets your inactivity timer. If you use your wallet even once within your configured window, the timer starts over.
3. **Permissionless execution.** When the inactivity window expires, anyone can trigger the redistribution. There is no central server, no special admin key, and no node network required. A "crank" operator (incentivized by a small fee) submits the redistribution transaction, and the Solana program executes it exactly as the vault owner configured.

4.2 What Deadswitch Is Not

To be very clear about what Deadswitch does NOT do:

- **It is not a wallet.** Deadswitch does not store, manage, or have access to your private keys. You continue using your existing wallet (Phantom, Backpack, Solflare, etc.) exactly as you do today.
- **It is not a custodian.** At no point does Deadswitch, its developers, or any third party have the ability to move your funds. Only the onchain program can execute redistribution, and only under the exact conditions you set.

- **It is not a legal will.** Deadswitch is a technical tool. It does not provide legal advice and does not replace proper estate planning. It complements traditional legal instruments by ensuring the crypto-specific part of inheritance actually works.
- **It is not a dead man's switch for secrets.** Unlike Sarcophagus, Deadswitch doesn't encrypt and reveal files. It directly transfers tokens. Your beneficiaries don't need to know anything about seed phrases, derivation paths, or wallet imports. They just receive tokens in their wallet.

4.3 The Core Idea in One Sentence

"If I stop using my wallet for 90 days, send 50% of my SOL to my wife, 30% to my brother, and 20% to a charity — automatically, trustlessly, and without anyone needing my seed phrase."

5. How It Works — The Simple Version

This section explains Deadswitch without any technical jargon. If you're a developer, skip to Section 6.

5.1 Step-by-Step User Journey

Step 1: Connect Your Wallet

You visit the Deadswitch app and connect your existing Solana wallet (Phantom, Backpack, Solflare, or any Solana-compatible wallet). Deadswitch does not create a new wallet for you — it works with the one you already use.

Step 2: Create a Vault

A "vault" is your inheritance plan. You set up three things:

1. **Which assets to include** — Pick which tokens you want to include in your plan. For example: "All my SOL and all my USDC." You can also include specific SPL tokens or set specific amounts.
2. **Who gets what** — Add your beneficiaries by entering their Solana wallet addresses and the percentage each person should receive. For example:
 - Wife (wallet address): 50%
 - Brother (wallet address): 30%
 - Charity (wallet address): 20%
3. **How long to wait** — Set your inactivity window. This is the number of days your wallet must be completely inactive before redistribution begins. We recommend 90 days as a reasonable default, but you can set it anywhere from 30 days to 365 days.

Step 3: Approve & Sign

You sign a single transaction that registers your vault on the Solana blockchain. This transaction:

- Records your beneficiaries and their percentages.
- Records your chosen inactivity window.
- Authorizes the Deadswitch program to transfer specific assets when — and only when — the conditions are met.
- This costs less than \$0.01 in Solana transaction fees.

Step 4: Live Your Life

That's it. There is nothing else you need to do. Deadswitch monitors your wallet activity in the background using Helius webhooks (a Solana infrastructure service). Every time you do anything with your wallet — send SOL, swap tokens, interact with a DeFi protocol, mint an NFT — the inactivity timer resets to zero.

You will never need to "check in" or "prove you're alive." Your normal wallet usage is your proof of life.

Step 5: What Happens If You Become Inactive

If your wallet has had zero activity for the full duration of your inactivity window (e.g., 90 days), the following process begins:

1. **Warning phase (optional):** If you provided contact information (email, Telegram), Deadswitch sends you alerts at 50%, 75%, and 90% of your inactivity window. For example, at day 45, day 67, and day 81 of a 90-day window. These are reminders to use your wallet if you're still active.
2. **Grace period:** After the full inactivity window passes, there is an additional grace period (default: 7 days) before redistribution can be executed. This is a final safety net.
3. **Redistribution:** After the grace period, anyone (a "crank operator") can submit a transaction to execute the redistribution. The Deadswitch program verifies onchain that the inactivity conditions are met, then automatically transfers the designated assets to your beneficiaries according to the percentages you set. Each beneficiary receives their share directly in their wallet — no action required on their end.

Step 6: What Your Beneficiaries Experience

Your beneficiaries don't need to do anything to "claim" their inheritance. They don't need to know about Deadswitch at all. They simply receive tokens in their Solana wallet, the same way they'd receive any normal transfer.

If your wife opens her Phantom wallet one day and sees an incoming transfer of SOL and USDC, that's it. She received her inheritance. She can use it, hold it, swap it, or send it to a bank via an offramp — all using her normal wallet.

5.2 Can I Change My Plan?

Yes. At any time while you are active, you can:

- **Add or remove beneficiaries.**
- **Change percentage splits.**
- **Change your inactivity window** (e.g., from 90 days to 180 days).
- **Add or remove assets** from the vault.
- **Cancel the vault entirely** — all tokens and permissions are returned to you, as if Deadswitch never existed.

All changes require your wallet signature, so no one else can modify your plan.

5.3 What If I Just Went on a Long Trip?

This is the most common concern, and it's addressed at multiple levels:

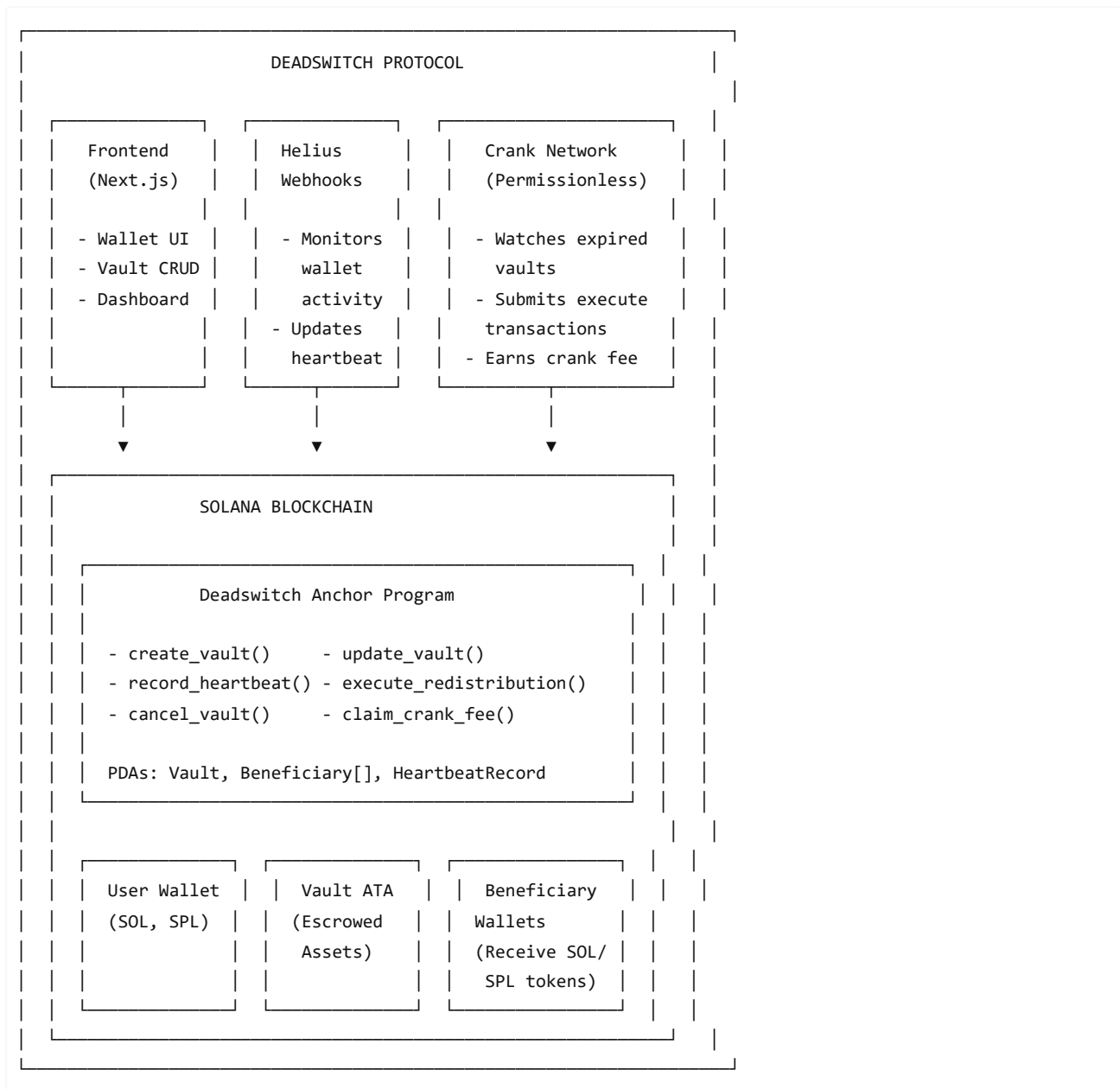
1. **You set the window.** If you travel for months at a time, set your window to 365 days instead of 90.
2. **Any activity resets it.** Even a tiny transaction (sending 0.001 SOL to yourself) resets the entire timer. You can do this from your phone in 5 seconds.
3. **Alerts warn you.** If you've set up notifications, you'll get warnings well before the window expires.
4. **Grace period.** Even after the window expires, the grace period gives you extra time to perform any activity.
5. **You can cancel anytime.** If you realize your window is too short, just update it.

6. Technical Architecture

This section is for developers, auditors, and technical reviewers.

6.1 System Overview

Deadswitch consists of four components:



Component 1: Anchor Program (Onchain)

The core protocol logic. A Solana program written in Anchor (Rust) that manages vault creation, heartbeat recording, and asset redistribution. This is the source of truth — everything else is a convenience layer on top.

Key properties:

- Immutable once deployed (upgradeable authority can be revoked post-audit).
- All state transitions are verified onchain.
- No admin keys or backdoors. The program authority is only used for upgrades during the development phase and will be burned after final audit.

Component 2: Helius Webhooks (Off-chain Monitoring)

Helius is a Solana infrastructure provider that offers webhook-based wallet monitoring. Deadswitch uses Helius to watch vault owners' wallets for any transaction activity.

How it works:

1. When a user creates a vault, the backend registers a Helius webhook for that wallet address.
2. Helius monitors the Solana blockchain and sends an HTTP callback whenever the wallet signs any transaction.
3. The backend receives this callback and submits a `record_heartbeat` transaction to the Deadswitch program, updating the vault's `last_activity` timestamp onchain.

Why Helius and not a custom node network:

- Helius already monitors over 100,000 wallet addresses at scale.
- No cold-start problem — the infrastructure is battle-tested and operational today.
- Cost: a few dollars per month for webhook subscriptions, not millions for a custom node network.
- Redundancy: if Helius goes down, the onchain timestamp is the authoritative check (see Section 9: Security Model).

Component 3: Crank Network (Permissionless Execution)

When a vault's inactivity window + grace period has elapsed, the redistribution doesn't happen automatically — someone needs to submit the `execute_redistribution` transaction. This is the "crank."

Anyone can be a crank operator. There is no special permission, staking requirement, or approval needed. You just call the `execute_redistribution` instruction on the Deadswitch program with a valid vault address. The program verifies onchain that all conditions are met before executing.

Crank incentive: The vault owner sets a small crank fee (default: 0.1% of redistributed assets, configurable) that is paid to whoever successfully submits the redistribution transaction. This creates a natural economic incentive for bots and individuals to monitor expired vaults and execute redistributions promptly.

Why this works:

- It's the same pattern used by Solana DeFi protocols (e.g., liquidation bots on Marginfi/Kamino, order execution on Jupiter Limit Orders).
- No trust required — the crank operator can't steal funds, change beneficiaries, or modify the vault. They can only trigger a redistribution that the owner already configured.
- Competitive market — multiple crank operators competing means vaults get executed quickly after expiry.

Component 4: Frontend Application (Off-chain UI)

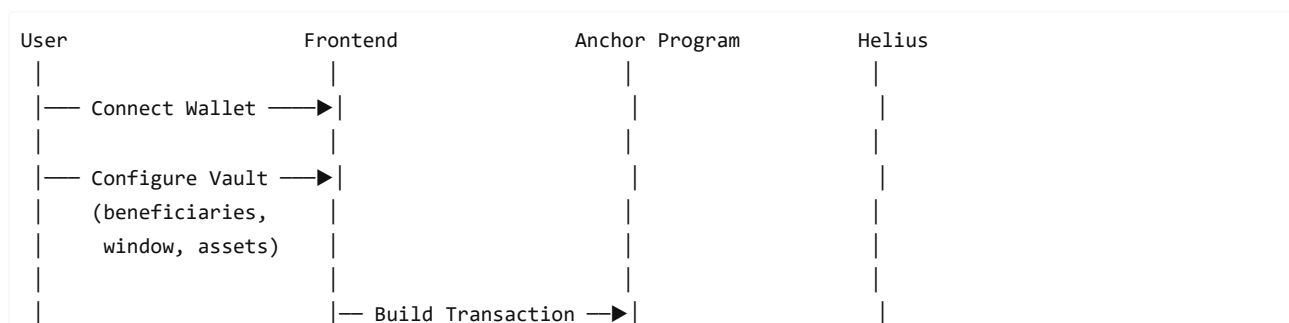
A Next.js web application that provides the user interface for creating and managing vaults. This is a convenience layer — all critical state lives onchain, and the protocol works even if the frontend goes offline.

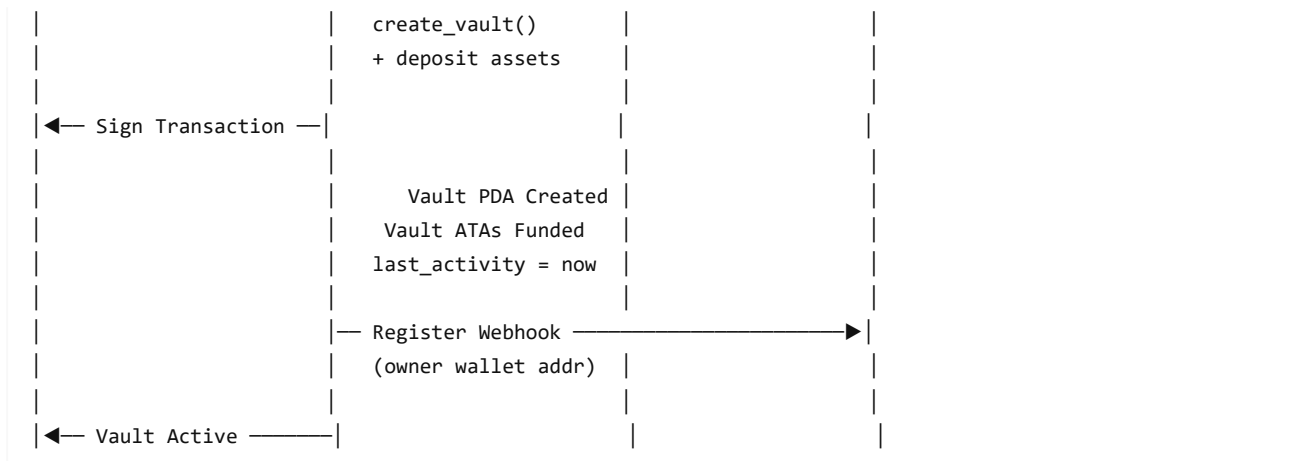
Stack:

- Next.js 15 + TypeScript (strict mode)
- Solana Wallet Adapter (Phantom, Backpack, Solflare, etc.)
- TailwindCSS for styling
- Server-side rendering for SEO and performance

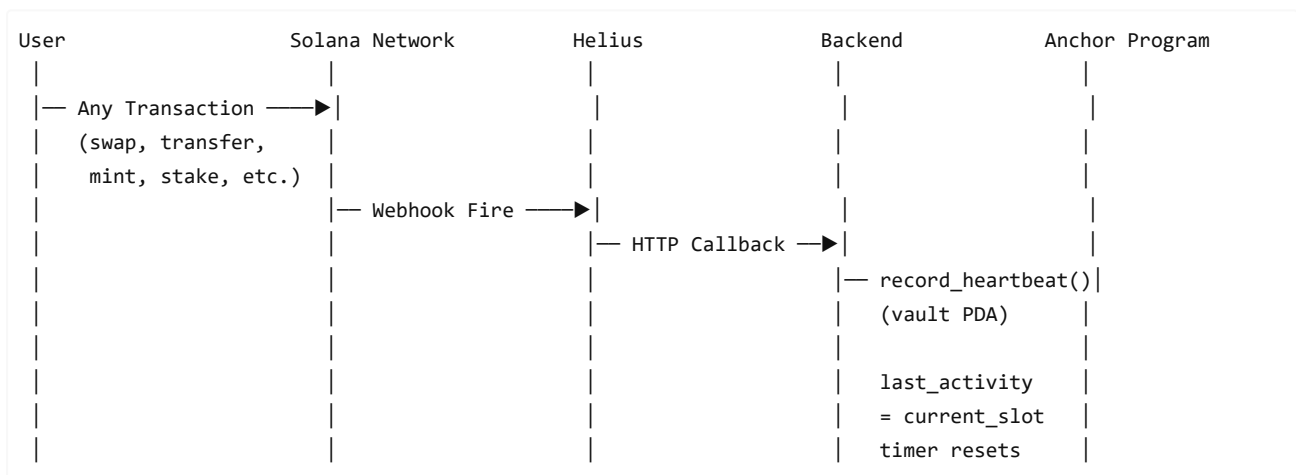
6.2 Data Flow

Creating a Vault

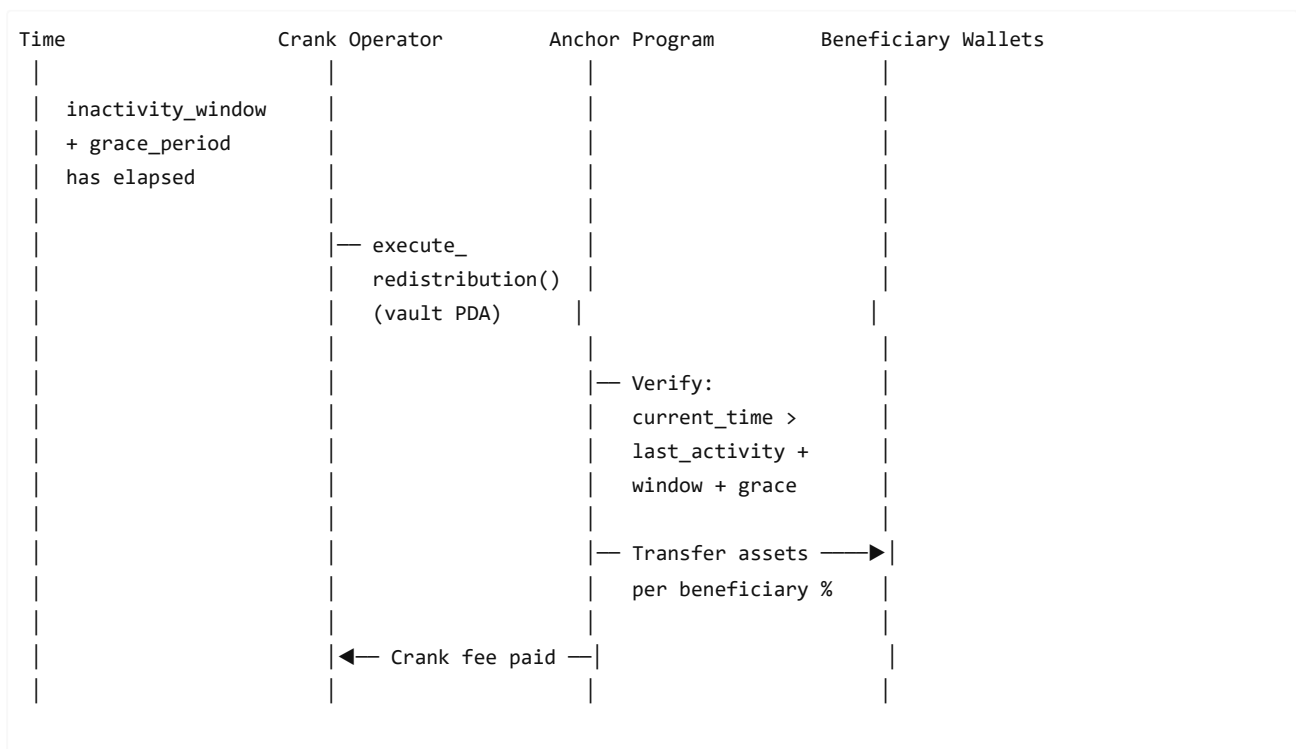




Heartbeat (Proof of Life)



Redistribution (After Inactivity)



		— Vault status =	
		Executed	

6.3 Tech Stack Summary

Layer	Technology	Purpose
Smart Contract	Anchor (Rust) on Solana	Core protocol logic, vault state, asset transfer
Wallet Monitoring	Helius Webhooks	Detect owner wallet activity, trigger heartbeats
Backend	Next.js API Routes + TypeScript	Webhook receiver, crank coordination, vault indexer
Frontend	Next.js 15 + React 19 + TypeScript	User interface for vault management
Wallet Integration	Solana Wallet Adapter	Connect Phantom, Backpack, Solflare, etc.
Database	PostgreSQL (Drizzle ORM)	Vault metadata cache, notification queue, crank logs
Notifications	Email (Resend) + Telegram Bot API	Inactivity alerts to vault owners
Hosting	Vercel (frontend) + VPS (backend/crank)	Application deployment

7. Smart Contract Design

7.1 Program Accounts

The Deadswitch Anchor program uses the following account structures:

Vault Account (PDA)

The primary account that stores all vault configuration and state.

```
Vault {
  // Identity
  owner: Pubkey,           // Wallet address of the vault creator
  vault_id: u64,           // Auto-incrementing vault ID for this owner
  bump: u8,                // PDA bump seed

  // Configuration
  inactivity_window: i64,  // Seconds of inactivity before trigger (e.g., 7,776,000 = 90 days)
  grace_period: i64,      // Additional seconds after window before execution allowed (e.g., 604,800
= 7 days)
  crank_fee_bps: u16,      // Fee paid to crank operator in basis points (e.g., 10 = 0.1%)

  // State
  status: VaultStatus,    // Active, Warning, Triggered, Executed, Cancelled
  last_activity: i64,     // Unix timestamp of last detected wallet activity
  created_at: i64,        // Unix timestamp of vault creation
  updated_at: i64,        // Unix timestamp of last vault modification

  // Assets
  asset_configs: Vec<AssetConfig>, // List of assets included in the vault
```

```

// Beneficiaries
beneficiaries: Vec<Beneficiary>, // List of beneficiaries and their share percentages

// Metadata
name: String,           // Human-readable vault name (max 64 chars)
note: String,           // Optional note/message to beneficiaries (max 256 chars)
}

```

VaultStatus Enum

```

VaultStatus {
    Active,           // Normal operation – timer is ticking, owner is active
    Warning,          // Inactivity window has passed, grace period active
    Triggered,        // Grace period has passed, ready for execution
    Executed,          // Redistribution has been completed
    Cancelled,         // Vault was cancelled by the owner
}

```

AssetConfig

```

AssetConfig {
    mint: Pubkey,           // SPL token mint address (Pubkey::default() = native SOL)
    amount_type: AmountType, // Fixed amount or percentage of balance
    amount: u64,            // Amount (in lamports/smallest unit) or basis points
}

AmountType {
    FixedAmount, // Transfer exactly this many tokens
    Percentage,  // Transfer this % of the vault's token balance (in bps, 10000 = 100%)
}

```

Beneficiary

```

Beneficiary {
    wallet: Pubkey,           // Beneficiary's Solana wallet address
    share_bps: u16,           // Share in basis points (e.g., 5000 = 50%)
    name: String,             // Human-readable name (max 32 chars, for UI display only)
}

```

7.2 PDA Derivation

All vault accounts are derived as Program Derived Addresses (PDAs) to ensure uniqueness and deterministic addressing.

```

Vault PDA seeds: ["vault", owner_pubkey, vault_id (as le_bytes)]
Vault ATA: Standard Associated Token Account for each SPL token, owned by the Vault PDA

```

This means:

- Each owner can create multiple vaults (identified by `vault_id`).
- Vault addresses are deterministic — anyone can compute the vault address from the owner's pubkey and vault ID.
- The Vault PDA is the authority over its own token accounts, enabling the program to transfer assets during redistribution without needing the owner's signature.

7.3 Instructions

`create_vault`

Creates a new vault with the specified configuration and funds it with the designated assets.

Signers: Vault owner **Accounts:** Owner wallet, Vault PDA (init), Vault ATAs (init), System Program, Token Program, Associated Token Program **Parameters:** `name` , `inactivity_window` , `grace_period` , `crank_fee_bps` , `asset_configs[]` , `beneficiaries[]` , `note`

Validation rules:

- `inactivity_window` must be between 30 days (2,592,000 seconds) and 365 days (31,536,000 seconds).
- `grace_period` must be between 1 day (86,400 seconds) and 30 days (2,592,000 seconds).
- `crank_fee_bps` must be between 1 (0.01%) and 500 (5%).
- Total `beneficiary.share_bps` must equal exactly 10,000 (100%).
- At least 1 beneficiary, maximum 10 beneficiaries.
- At least 1 asset config, maximum 20 asset configs.
- All beneficiary wallet addresses must be valid (not the zero address, not the owner's address).
- `name` max 64 characters, `note` max 256 characters.

`record_heartbeat`

Updates the vault's `last_activity` timestamp. Called by the backend when Helius detects wallet activity.

Signers: Heartbeat authority (backend wallet) OR vault owner **Accounts:** Vault PDA **Parameters:** `activity_timestamp` , `activity_signature` (the Solana transaction signature that triggered the heartbeat)

Validation rules:

- Vault status must be `Active` or `Warning` .
- `activity_timestamp` must be more recent than the current `last_activity` .
- `activity_timestamp` must not be in the future (within a 30-second tolerance for clock drift).
- If vault was in `Warning` status, it transitions back to `Active` .

`update_vault`

Allows the vault owner to modify vault configuration.

Signers: Vault owner **Accounts:** Vault PDA, (optionally) additional ATAs if adding new assets **Parameters:** Any combination of: `name` , `inactivity_window` , `grace_period` , `crank_fee_bps` , `asset_configs[]` , `beneficiaries[]` , `note`

Validation rules:

- Same validation as `create_vault` for all fields.
- Vault status must be `Active` or `Warning` .
- Cannot update a `Triggered` , `Executed` , or `Cancelled` vault.
- Updating the vault also resets `last_activity` to the current timestamp (since updating requires the owner's signature, which proves they're alive).

`top_up_vault`

Allows the owner to deposit additional assets into an existing vault.

Signers: Vault owner **Accounts:** Vault PDA, Owner token accounts, Vault ATAs **Parameters:** `asset_configs[]` (assets and amounts to add)

Validation rules:

- Vault status must be `Active` or `Warning` .
- Also resets `last_activity` to current timestamp.

`execute_redistribution`

Transfers vault assets to beneficiaries. Can be called by anyone (permissionless crank).

Signers: Any wallet (the crank operator) **Accounts:** Vault PDA, Vault ATAs, all Beneficiary wallet addresses and their ATAs, Crank operator wallet **Parameters:** None (everything is read from the vault state)

Validation rules:

- Vault status must be `Active` or `Warning` (the instruction itself checks timing).
- `current_timestamp > last_activity + inactivity_window + grace_period` .
- All beneficiary accounts must be provided and match the vault's beneficiary list.
- On success:
 - Each asset is split according to beneficiary `share_bps` .
 - Transfers are executed from Vault ATAs to beneficiary wallets/ATAs.
 - Crank fee (in each asset, proportional) is sent to the crank operator.
 - Vault status is set to `Executed` .
 - Vault ATAs are closed, rent returned to the vault owner.

`cancel_vault`

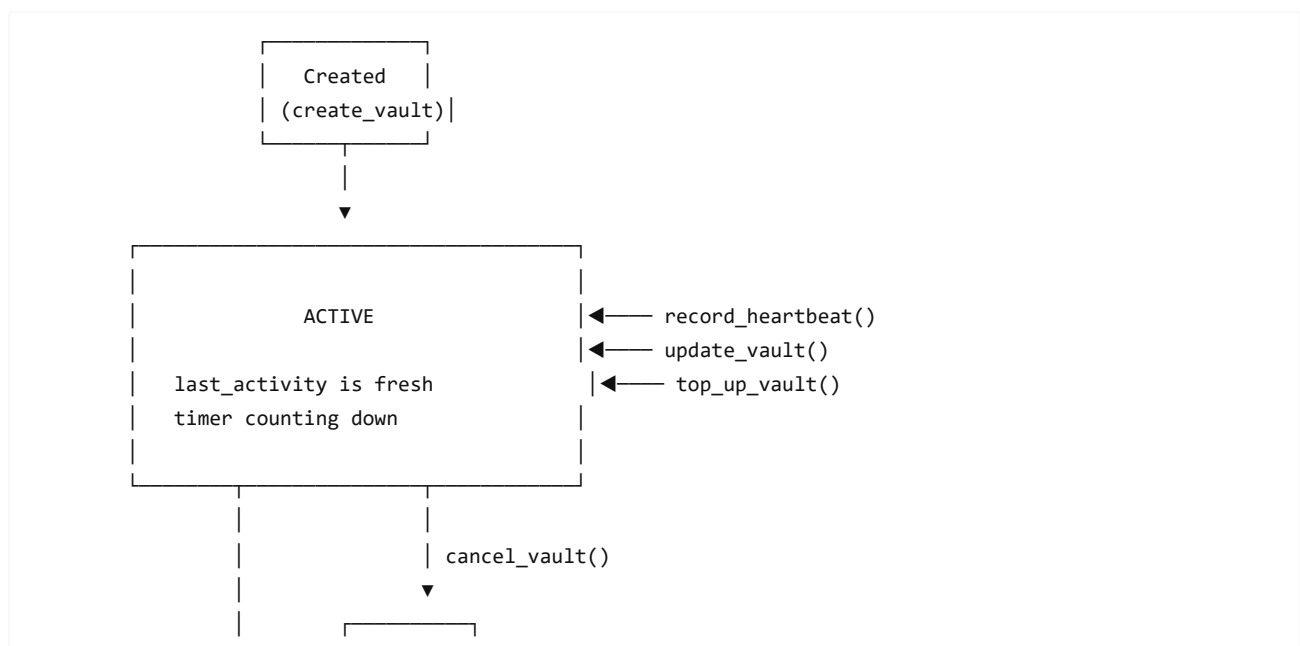
Cancels a vault and returns all assets to the owner.

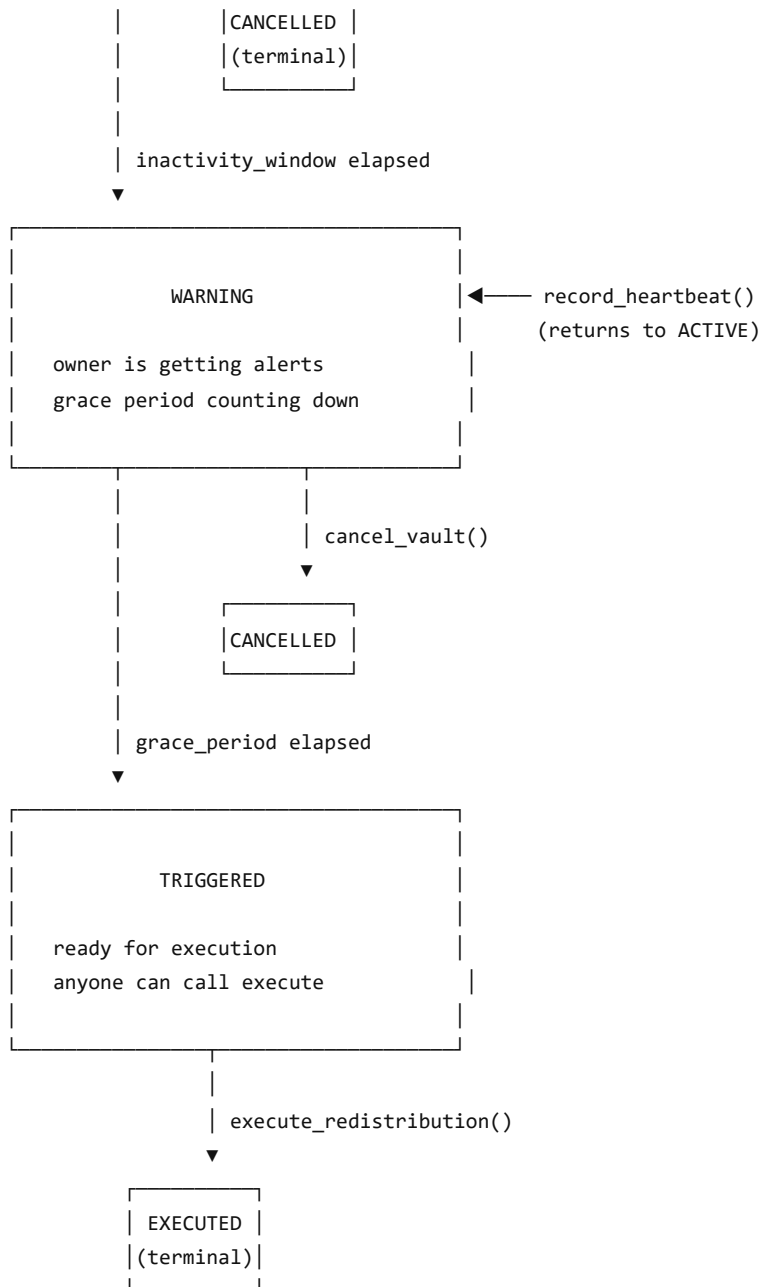
Signers: Vault owner **Accounts:** Vault PDA, Vault ATAs, Owner wallet **Parameters:** None

Validation rules:

- Vault status must be `Active` or `Warning` .
- Cannot cancel a vault that is already `Triggered` , `Executed` , or `Cancelled` .
- All assets in Vault ATAs are transferred back to the owner.
- Vault ATAs are closed, rent returned to owner.
- Vault status is set to `Cancelled` .

7.4 State Machine





Important: The `Warning` and `Triggered` states are not explicitly stored onchain as state transitions. Instead, the program computes the effective status based on timestamps:

```

if status == Cancelled || status == Executed:
    return status // terminal states

elapsed = current_time - last_activity

if elapsed < inactivity_window:
    return Active
else if elapsed < inactivity_window + grace_period:
    return Warning
else:
    return Triggered
  
```

This means the vault's status is always accurate without needing someone to submit a state-transition transaction. The onchain `status` field is only set explicitly for the terminal states (`Executed` and `Cancelled`).

8. Protocol Mechanics

8.1 Inactivity Detection

What Counts as "Activity"

Any transaction **signed by the vault owner's wallet** counts as proof of life. This includes:

- Sending SOL or SPL tokens to any address
- Swapping tokens on Jupiter, Raydium, or any DEX
- Staking or unstaking SOL
- Minting or burning NFTs
- Interacting with any DeFi protocol (Marginfi, Kamino, Marinade, etc.)
- Voting in governance
- Even a self-transfer (sending 0.001 SOL to yourself)

What does **NOT** count:

- Receiving tokens (someone sending you SOL doesn't prove you're alive)
- Airdrops landing in your wallet
- Token account rent changes
- Validator rewards (if staking)

The distinction is simple: **did the owner sign a transaction?** If yes, they're alive. If no, the timer keeps ticking.

How Monitoring Works

1. When a vault is created, the backend registers a **Helius Enhanced Transaction Webhook** for the owner's wallet address.
2. Helius monitors the Solana blockchain in real-time (sub-second latency).
3. When the owner signs any transaction, Helius sends an HTTP POST to the Deadswitch backend with transaction details.
4. The backend validates the webhook payload (signature verification, timestamp checks) and submits a `record_heartbeat` transaction to the Deadswitch program.
5. The onchain `last_activity` timestamp is updated to the time of the detected transaction.

Heartbeat Authority

The backend wallet that submits `record_heartbeat` transactions is called the **heartbeat authority**. This is a special wallet address that is registered in the Deadswitch program as authorized to update heartbeats.

Important security note: The heartbeat authority can only update timestamps — it cannot modify vault configuration, cancel vaults, or trigger redistributions. It is a write-only role with a very narrow scope.

Additionally, the vault owner themselves can always call `record_heartbeat` directly (e.g., via the frontend), bypassing the backend entirely. This provides a manual fallback.

8.2 Inactivity Window Configuration

Parameter	Minimum	Default	Maximum
Inactivity Window	30 days	90 days	365 days
Grace Period	1 day	7 days	30 days

Choosing the Right Window

- **30 days:** For highly active traders who transact daily. Not recommended unless you're certain you'll never take a month-long break from crypto.
- **90 days (recommended):** Balances safety and responsiveness. Most active crypto users transact at least once every 3 months.
- **180 days:** For long-term holders who check in occasionally. Good for people who buy and hold.
- **365 days:** Maximum caution. Only use this if you routinely go many months without touching your wallet.

Grace Period Purpose

The grace period is an additional buffer **after** the inactivity window has fully elapsed. It serves two purposes:

1. **False positive protection:** If you were simply on a long trip or temporarily unable to access your wallet, the grace period gives you extra time.
2. **Notification window:** During the grace period, you receive final alerts (if configured) urging you to perform any wallet activity.

8.3 Multi-Channel Alerts

When a vault enters the warning zone (approaching inactivity threshold), Deadswitch sends alerts through multiple channels:

Trigger Point	Alert Type	Channel
50% of window elapsed	Gentle reminder	Email
75% of window elapsed	Stronger warning	Email + Telegram
90% of window elapsed	Urgent alert	Email + Telegram + Onchain event
Window elapsed (grace period starts)	Critical alert	All channels, repeated daily

Alert configuration is optional. If you don't provide contact information, no alerts are sent — the protocol still works entirely based on onchain timestamps.

Privacy consideration: Alert contact information (email, Telegram ID) is stored in the backend database, NOT onchain. The onchain vault contains no personally identifiable information.

8.4 Crank Mechanism

How Cranks Work

After a vault's inactivity window + grace period has fully elapsed, the vault is in `Triggered` state and ready for execution. At this point, any wallet can call `execute_redistribution`.

In practice, **crank operators** are automated bots that:

1. Periodically scan all Deadswitch vaults (using onchain data or an indexer).
2. Identify vaults where `current_time > last_activity + inactivity_window + grace_period`.
3. Submit `execute_redistribution` transactions for eligible vaults.
4. Collect the crank fee as payment.

Crank Economics

The crank fee is set by the vault owner (default 0.1%, configurable from 0.01% to 5%). The fee is deducted proportionally from each asset being redistributed.

Example: A vault contains 100 SOL and 10,000 USDC, with a 0.1% crank fee and two beneficiaries (50/50 split).

Recipient	SOL	USDC
Beneficiary A (50%)	49.95	4,997.50
Beneficiary B (50%)	49.95	4,997.50
Crank Operator (0.1%)	0.10	5.00
Total	100.00	10,000.00

Why This Incentive Works

- For large vaults (\$100K+), even 0.1% is \$100+ — more than enough to cover gas and make cranking profitable.
- For small vaults, higher crank fees (1-5%) may be needed to attract operators, but the amounts are small in absolute terms.
- Competition among crank operators ensures vaults are executed quickly — the first to submit wins the fee.
- No staking, approval, or registration required — pure economic incentive.

8.5 Asset Handling

Supported Assets

Deadswitch supports:

1. **Native SOL** — wrapped into wSOL for programmatic handling, then unwrapped for beneficiaries.
2. **Any SPL Token** — USDC, USDT, BONK, JUP, any token with a valid mint address.
3. **Compressed NFTs (cNFTs)** — future roadmap (v2).
4. **Staked SOL** — future roadmap (v2, requires unstaking flow).

Deposit Model: Escrow vs. Approval

Deadswitch uses an **escrow model**: when you create a vault, your designated assets are transferred into the vault's token accounts (ATAs owned by the Vault PDA). This means:

Pros:

- Guaranteed execution — the assets are already in the vault, so redistribution always succeeds.
- No risk of the owner spending the designated assets before redistribution.
- Simpler smart contract logic — no need to check balances at execution time.

Cons:

- Assets are locked in the vault until redistribution or cancellation.
- The owner cannot use these specific assets for trading, staking, or DeFi while they're in the vault.

Why escrow over approval: An approval-based model (where assets stay in the owner's wallet and the program has permission to pull them at execution time) is more flexible but less reliable. If the owner spends the assets before the trigger fires, the beneficiaries get nothing. For an inheritance protocol, reliability is more important than flexibility. You can always cancel the vault and re-create it if you want to rebalance.

Percentage-Based Assets

For assets configured with `AmountType::Percentage`, the vault doesn't lock a fixed amount. Instead, at the time of `create_vault` or `top_up_vault`, the user deposits tokens. At redistribution time, the total balance of each token in the vault is split among beneficiaries.

This means users can top up the vault over time, and the redistribution will always cover whatever is in the vault at the time of execution.

9. Security Model

9.1 Threat Model

Threat 1: False Trigger (Premature Redistribution)

Scenario: The owner is alive and active, but the system incorrectly triggers redistribution.

Mitigations:

- The inactivity window is enforced **onchain** by comparing `Clock::get().unix_timestamp` with `vault.last_activity`. No off-chain system can override this check.
- The grace period provides an additional buffer.
- Multi-channel alerts notify the owner well before the trigger point.
- Even if Helius webhooks fail to detect activity, the owner can manually call `record_heartbeat` via the frontend (or directly via CLI) to reset the timer.
- The minimum inactivity window is 30 days — accidental triggers from short outages are impossible.

Residual risk: Extremely low. Would require the owner to be alive, active, aware, and still unable to submit a single transaction for 30+ days — a near-impossible scenario.

Threat 2: Helius Downtime

Scenario: Helius webhooks stop working, causing heartbeats to stop updating even though the owner is active.

Mitigations:

- The backend implements health checks on Helius webhook delivery. If no webhooks are received for any monitored wallet within 24 hours, an alert is raised.
- The owner can always submit a manual heartbeat (via the frontend or CLI).
- The inactivity window + grace period provides a large buffer — even weeks of Helius downtime wouldn't cause a false trigger for most configurations.
- Future enhancement: add redundant monitoring via multiple providers (Alchemy, QuickNode, direct RPC polling).

Threat 3: Crank Griefing

Scenario: A malicious actor tries to execute redistribution on vaults that aren't eligible.

Mitigation: The `execute_redistribution` instruction performs a strict onchain timestamp check. If the conditions aren't met, the transaction simply fails. The attacker wastes gas on a reverted transaction. No state is modified.

Threat 4: Backend Compromise

Scenario: An attacker gains access to the backend server and the heartbeat authority wallet.

Impact: The attacker could:

- **Stop recording heartbeats** — this doesn't immediately cause harm, as vaults have long inactivity windows. The team would notice and remediate within hours/days.
- **Submit false heartbeats** — this would keep vaults alive (prevent redistribution) even if the owner is truly inactive. While undesirable, this is a liveness failure (delayed execution), not a safety failure (premature execution or theft).

What the attacker CANNOT do:

- Steal assets from any vault (only the program can transfer vault assets, and only during a valid redistribution).
- Modify vault configuration (requires the owner's signature).
- Cancel vaults (requires the owner's signature).

- Redirect assets to different beneficiaries (beneficiary list is stored onchain, set by the owner).

Threat 5: Smart Contract Bug

Scenario: A bug in the Anchor program allows unauthorized asset transfers.

Mitigations:

- The program will be audited by a professional Solana auditing firm before mainnet launch.
- The program is open-source, allowing community review.
- Upgrade authority will be retained during early mainnet operation (to fix critical bugs) and then burned after a stability period.
- A bug bounty program will be established post-launch.

Threat 6: Owner Wants to Cancel But Can't Access Wallet

Scenario: The owner's wallet is compromised or they lose access, but they want to prevent redistribution.

Mitigation: This is an inherent limitation. If the owner can't sign transactions with their wallet, they can't interact with Deadswitch — or any other Solana protocol. This is a wallet security issue, not a Deadswitch issue. We recommend users secure their wallets with hardware wallets (Ledger) and standard security practices.

9.2 Security Principles

1. **Onchain verification is the source of truth.** All timing checks happen in the Anchor program using Solana's `Clock` sysvar. No off-chain system can override these checks.
2. **Minimum privilege.** Each role (owner, heartbeat authority, crank operator) has the minimum permissions needed:
 - Owner: full control (create, update, cancel, heartbeat).
 - Heartbeat authority: can only update timestamps.
 - Crank operator: can only trigger valid redistributions.
3. **Fail-safe over fail-fast.** If any off-chain component fails (Helius, backend, notifications), the protocol defaults to NOT redistributing. A missed heartbeat extends the time before redistribution, it doesn't accelerate it.
4. **No admin backdoor.** There is no admin key that can drain vaults, override configurations, or bypass timing checks. The upgrade authority exists only for program upgrades and will be burned after the stability period.

9.3 Known Limitations

We believe in transparency about what Deadswitch cannot do:

Limitation	Explanation
Cannot protect against wallet compromise	If someone steals your private key, they can cancel your vault and steal your assets — same as any crypto protocol.
Off-chain notifications may fail	Email and Telegram alerts are best-effort. The protocol works without them, but you might miss warnings.
Crank execution is not instant	After a vault triggers, someone needs to submit the crank transaction. In practice, this should happen within minutes (due to economic incentives), but it's not guaranteed to be immediate.
Cannot handle non-SPL assets	NFTs (non-compressed), staked SOL positions, and LP tokens require specialized handling — planned for v2.

Legal status is unclear

Onchain inheritance may or may not be legally recognized in your jurisdiction. Deadswitch is a technical tool, not a legal instrument. Consult a lawyer.

10. User Experience

10.1 Design Philosophy

Deadswitch is designed for two distinct audiences:

1. **Vault owners** — crypto-native individuals who understand wallets and tokens. The setup process can involve some complexity (choosing windows, configuring beneficiaries), but should be intuitive and guided.
2. **Beneficiaries** — potentially non-crypto-native individuals (family members, partners). Their experience must be zero-effort. They should receive tokens in their wallet without needing to understand Deadswitch, smart contracts, or blockchain mechanics.

Guiding principles:

- **Setup should take under 5 minutes.** Connect wallet → name your vault → add beneficiaries → set window → deposit → sign → done.
- **After setup, the owner should forget about it.** No check-ins, no maintenance, no "keep alive" buttons. Just live your life.
- **Beneficiaries should feel like they received a normal transfer.** No claim pages, no seed phrase imports, no protocol interactions.

10.2 Owner Experience

Dashboard

After connecting their wallet, vault owners see a dashboard showing:

- **Active vaults** — name, status (Active/Warning), time since last activity, time remaining before trigger, total value locked.
- **Quick actions** — create new vault, top up existing vault, update vault, cancel vault.
- **Activity log** — history of heartbeats (detected wallet activity), with timestamps and transaction signatures.
- **Alert settings** — configure email and/or Telegram for inactivity warnings.

Vault Creation Flow

A guided, step-by-step wizard:

Step 1 — Name Your Vault "Give your vault a name (e.g., 'Family Inheritance', 'Emergency Fund')." Optional: add a note/message for your beneficiaries.

Step 2 — Add Beneficiaries "Who should receive your assets?"

- Enter wallet address (or scan QR code)
- Enter name (for your reference)
- Enter percentage share
- [+ Add another beneficiary]
- Visual pie chart showing the split
- Validation: percentages must total 100%

Step 3 — Choose Your Assets "Which assets do you want to include?"

- Shows all tokens in the connected wallet with balances
- User selects tokens and amounts (or "All" for full balance)

- Shows estimated USD value

Step 4 — Set Your Timing "How long should your wallet be inactive before redistribution?"

- Slider or dropdown: 30 / 60 / 90 / 180 / 365 days
- Recommendation text based on usage patterns
- Grace period: auto-set to 7 days (advanced users can modify)
- Crank fee: auto-set to 0.1% (advanced users can modify)

Step 5 — Review & Confirm

- Summary of everything: vault name, beneficiaries + shares, assets + amounts, timing
- Estimated gas cost (typically < \$0.01)
- One-click sign and submit

10.3 Beneficiary Experience

The beneficiary experience is intentionally minimal:

1. **Before redistribution:** The beneficiary knows nothing. They may not even know they've been designated as a beneficiary. (The vault owner may choose to inform them, but the protocol doesn't require it.)
2. **During redistribution:** The crank operator submits the execution transaction. The beneficiary's wallet receives incoming token transfers — SOL, USDC, or whatever assets the vault contained.
3. **After redistribution:** The beneficiary sees the tokens in their wallet, just like any other incoming transfer. If they use Phantom, they see it in their transaction history. If they use a portfolio tracker, it shows up as a received asset.

No claim page. No app download. No seed phrase. No protocol interaction. Just tokens arriving in their wallet.

Optional: Beneficiary Notification

If the vault owner provided beneficiary contact information (email or Telegram), Deadswitch can send a notification after redistribution:

"You have received an inheritance transfer via Deadswitch. [X] SOL and [Y] USDC have been sent to your wallet [address]. These assets are yours — you can use them immediately with your Solana wallet. If you need help, visit deadswitch.xyz/help."

This is optional and purely informational.

10.4 Mobile Experience

The Deadswitch web app is fully responsive and works in mobile browsers. Key mobile considerations:

- Vault creation works via Phantom/Backpack in-app browser.
- Dashboard is optimized for small screens (card layout, collapsible sections).
- Beneficiary QR code scanning uses the device camera.
- Alert notifications work via Telegram (native push notifications) and email.

11. Business Model

11.1 Revenue Streams

Deadswitch generates revenue through three channels:

Stream 1: Protocol Fee (Primary)

A small annual fee on assets locked in vaults, charged at the time of vault creation or renewal.

Vault Value (TVL per vault)	Annual Fee
< \$1,000	Free
\$1,000 – \$10,000	Free (growth phase)
\$10,000 – \$100,000	0.1% per year
\$100,000 – \$1,000,000	0.08% per year
\$1,000,000+	0.05% per year (negotiable)

Example: A vault holding \$50,000 in assets pays \$50/year. This is less than the cost of a single meeting with an estate lawyer.

How it's collected: The protocol fee is deducted from vault assets at the time of redistribution (in addition to the crank fee). If the vault is cancelled before redistribution, no protocol fee is charged (only gas costs for the cancellation transaction).

Stream 2: Premium Features (Secondary)

Feature	Free Tier	Premium (\$5/month)
Vaults per wallet	1	Unlimited
Beneficiaries per vault	3	10
Assets per vault	5	20
Email alerts	Basic (1 alert)	Full (50%, 75%, 90%, daily)
Telegram alerts	No	Yes
Custom branding on beneficiary notifications	No	Yes
API access	No	Yes
Priority crank execution	No	Yes

Stream 3: B2B / Institutional (Future)

For DAOs, funds, and crypto businesses that need treasury inheritance planning:

- Custom inactivity rules (e.g., multi-signer thresholds, committee approval)
- Compliance reporting and audit logs
- SLA-backed crank execution (guaranteed execution within 1 hour of trigger)
- White-label integration for wallet providers
- Pricing: \$100 – \$5,000/month depending on TVL and features

11.2 Revenue Projections

Conservative estimates based on crypto market growth and adoption:

Metric	Year 1	Year 2	Year 3
Total Vaults Created	1,000	10,000	50,000
Average TVL per Vault	\$5,000	\$15,000	\$25,000

Total Value Locked	\$5M	\$150M	\$1.25B
Protocol Fee Revenue	\$2,500	\$120,000	\$875,000
Premium Subscriptions	\$6,000	\$120,000	\$600,000
B2B Revenue	\$0	\$50,000	\$500,000
Total Annual Revenue	\$8,500	\$290,000	\$1,975,000

These projections assume only Solana. Multi-chain expansion (EVM, Bitcoin L2s) would multiply the addressable market significantly.

11.3 Unit Economics

- **Cost to serve one vault:** Near zero. Helius webhook monitoring costs ~\$0.001/day per wallet. Solana gas for heartbeats is ~\$0.001 per transaction. A vault with weekly heartbeats costs ~\$0.06/year to monitor.
- **Crank costs:** Covered by the crank fee (paid from vault assets to the crank operator, not by Deadswitch).
- **Infrastructure costs:** Backend server (\$20/month), database (\$10/month), domain (~\$15/year). Total fixed costs under \$400/year.
- **Gross margin:** >95% at scale.

12. Market Analysis

12.1 Total Addressable Market

The Crypto Inheritance Problem

- **Global crypto holders:** Estimated 560+ million people worldwide hold cryptocurrency as of 2025. [5]
- **Lost crypto:** An estimated \$140+ billion in crypto is permanently inaccessible due to lost keys, forgotten wallets, and deceased holders. [1]
- **Estate planning gap:** Only ~5% of crypto holders have any form of crypto-specific inheritance plan. [6]
- **Growing problem:** As early crypto adopters (2010s era) age, the inheritance crisis will accelerate dramatically over the next 10-20 years.

Market Sizing

Segment	Size	Deadswitch Relevance
Global crypto holders	560M+ people	Total addressable market
Holders with >\$1K in crypto	~100M people	Realistic target (worth insuring)
Holders with >\$10K in crypto	~20M people	Paying tier target
Annual crypto inheritance losses	~\$10B+ (est.)	Problem size Deadswitch addresses
Traditional estate planning market	\$3B+/year (US alone)	Adjacent market for crypto-native estate tools

Solana-Specific Market

- **Solana active wallets:** 10M+ monthly active wallets (2025). [7]
- **Solana DeFi TVL:** \$8B+ (2025). [7]
- **Inheritance tools on Solana:** Zero. Deadswitch would be the first and only option.

12.2 Competitive Landscape

Solution	Chain	Type	Status	Key Weakness
Sarcophagus	Ethereum	Decentralized dead man's switch	Low adoption	Reveals secrets, not assets. Cold-start problem. Manual check-in.
Casa Inheritance	Bitcoin	Centralized custody	Active	Custodial. Limited to Bitcoin. Expensive (\$250+/year).
Safe (Gnosis) Multisig	EVM	Multi-signature wallet	Active	Not designed for inheritance. Requires technical coordination.
Seed phrase on paper	Any	Manual process	Common	Insecure, fragile, requires technical knowledge from heir.
Lawyer + legal will	N/A	Legal instrument	Common	Cannot execute crypto transfers. Courts can't access wallets.
Deadswitch	Solana	Non-custodial inheritance protocol	Building	—

Deadswitch's Moat

1. **First mover on Solana** — no inheritance tools exist on Solana today.
2. **Asset transfer, not secret reveal** — fundamentally different from Sarcophagus.
3. **No node network** — no cold-start problem, leverages existing Helius infrastructure.
4. **Passive proof-of-life** — no manual check-ins, any wallet activity counts.
5. **Composable** — can be integrated into any Solana wallet or DeFi protocol as a feature.

12.3 Why Solana

Factor	Solana	Ethereum	Bitcoin
Transaction cost	~\$0.001	~\$1-50	~\$1-30
Transaction speed	~400ms	~12 sec	~10 min
Heartbeat cost (annual, weekly)	~\$0.05	~\$50-2,500	N/A
Smart contract capability	Full (Anchor/Rust)	Full (Solidity)	Limited (Script)
Existing inheritance tools	None	Sarcophagus (struggling)	Casa (custodial)
Ecosystem gap	Yes	Partial	Partial

Solana's sub-cent transaction costs make Deadswitch economically viable at any scale. On Ethereum, the heartbeat mechanism alone would cost more than the protocol fee. On Solana, monitoring a vault for a year costs less than a penny.

13. Roadmap

Phase 1: Hackathon MVP (April 13 – May 11, 2026)

Goal: Deliver a functional, demo-ready prototype on Solana devnet.

Feature	Priority	Status
---------	----------	--------

Anchor program: create_vault, cancel_vault, execute_redistribution	Must have	Planned
Anchor program: record_heartbeat	Must have	Planned
Frontend: wallet connect, vault creation wizard	Must have	Planned
Frontend: dashboard (vault list, status, activity log)	Must have	Planned
Helius webhook integration (heartbeat monitoring)	Must have	Planned
Basic crank bot (single operator)	Must have	Planned
SOL + USDC support	Must have	Planned
Devnet deployment + demo video	Must have	Planned
Email alerts (basic)	Nice to have	Planned
Mobile responsive UI	Nice to have	Planned

Deliverables for hackathon submission:

- Working devnet deployment
- 3-5 minute demo video showing full flow (create vault → simulate inactivity → redistribution)
- Open-source GitHub repository
- This whitepaper

Phase 2: Post-Hackathon Polish (June – August 2026)

Feature	Description
Security audit	Professional Solana program audit (OtterSec, Neodyme, or equivalent)
Multi-asset support	Any SPL token (not just SOL/USDC)
Telegram alerts	Bot integration for push notifications
Crank network	Open-source crank bot + documentation for anyone to run
UI/UX polish	Design overhaul, animations, onboarding tutorial
Analytics dashboard	TVL tracking, vault statistics, protocol health metrics

Phase 3: Mainnet Launch (September – November 2026)

Feature	Description
Mainnet deployment	Program deployment with upgrade authority (retained for bug fixes)
Premium tier launch	Subscription billing via Stripe/crypto
Multi-vault support	Multiple vaults per wallet
Beneficiary notifications	Email + Telegram notifications to beneficiaries after redistribution
API launch	REST API for third-party integrations
Bug bounty program	Immunefi or custom program

Phase 4: Growth & Expansion (2027+)

Feature	Description
Wallet integrations	Native Deadswitch features inside Phantom, Backpack, Solflare
cNFT / NFT support	Include compressed NFTs and standard NFTs in vaults
Staked SOL support	Handle staked positions (auto-unstake before redistribution)
LP token support	Handle DeFi LP positions (auto-withdraw before redistribution)
Multi-chain expansion	EVM chains (Ethereum, Base, Arbitrum) via cross-chain messaging
B2B / institutional tier	DAO treasury inheritance, multi-signer vaults, compliance features
Legal partnerships	Partnerships with estate lawyers and financial advisors
Upgrade authority burn	Revoke program upgrade authority after stability period

14. Team

Wira — Founder & Lead Developer

Full-stack developer with deep experience building production-grade applications on Solana and across the web stack. Proven track record shipping non-custodial blockchain applications from architecture to deployment, with a focus on security, code quality, and user experience.

Technical expertise:

- Solana program development (Anchor/Rust, SPL tokens, PDAs, CPIs)
- Full-stack TypeScript (Next.js, NestJS, React, Node.js)
- Backend systems (Go, PostgreSQL, Redis, WebSocket)
- DevOps & infrastructure (VPS deployment, PM2, Nginx, CI/CD)
- Security hardening (OWASP, rate limiting, input validation, auth flows)

15. References

- [1] Chainalysis, "Cryptocurrency Lost and Found," 2023 report on permanently inaccessible Bitcoin.
- [2] Chainalysis, "The State of Crypto," annual report on cryptocurrency adoption and dormant wallets.
- [3] Forbes, "Crypto Billionaire Matthew Mellon Dies, \$500M in XRP Left in Limbo," 2018.
- [4] The Wall Street Journal, "QuadrigaCX: A \$190 Million Crypto Mystery," 2019.
- [5] Triple-A, "Global Crypto Adoption Statistics," 2025 update.
- [6] Cremation Institute / Finder survey on crypto holders' estate planning practices, 2023.
- [7] Solana Foundation, "Solana Ecosystem Report," Q1 2025.

Appendix A: Glossary

Term	Definition
Anchor	A framework for building Solana programs (smart contracts) in Rust. Provides safety features and developer tools.
ATA (Associated Token Account)	A deterministic token account derived from a wallet address and token mint. Standard way to hold SPL tokens on Solana.
Basis Points (bps)	One hundredth of a percent. 100 bps = 1%. 10,000 bps = 100%. Used for precise percentage calculations.
Crank	An external actor who submits a transaction to trigger a protocol action. Common pattern in Solana DeFi.
Dead Man's Switch	A mechanism that activates when the operator fails to take an expected action (e.g., pressing a button, signing a transaction) within a defined period.
Escrow	Holding assets in a third-party account (in this case, a program-controlled account) until conditions are met.
Helius	A Solana infrastructure provider offering RPC nodes, webhooks, and data APIs.
Heartbeat	A signal confirming the vault owner is still active. In Deadswitch, any signed wallet transaction serves as a heartbeat.
Lamports	The smallest unit of SOL (1 SOL = 1,000,000,000 lamports). Similar to how cents are the smallest unit of a dollar.
PDA (Program Derived Address)	A special Solana account address derived deterministically from seeds and a program ID. Cannot have a private key — only the program can sign for it.
SPL Token	Solana's standard for fungible tokens (like ERC-20 on Ethereum). USDC, USDT, BONK, and JUP are all SPL tokens.
TVL (Total Value Locked)	The total dollar value of assets deposited in a protocol.

Appendix B: FAQ

Q: What happens if Deadswitch (the company) shuts down? A: The onchain program continues to work forever, regardless of whether the company exists. Vaults, heartbeats, and redistributions are all executed by the Solana program — not by our servers. The only features that would stop working are off-chain notifications (email/Telegram alerts) and the hosted frontend. However, the frontend is open-source, so anyone can host it, and users can interact with the program directly via CLI.

Q: Can I use Deadswitch with a hardware wallet (Ledger)? A: Yes. Deadswitch works with any Solana-compatible wallet, including Ledger via Phantom/Solflare integration.

Q: What if I want to change my beneficiaries after creating a vault? A: You can update your vault at any time by signing an `update_vault` transaction. You can add, remove, or change beneficiaries and their percentage splits.

Q: Can my beneficiaries see my vault before redistribution? A: The vault data is stored onchain (public by nature of blockchain). However, beneficiary names and contact information are stored off-chain. A technically savvy person could find the vault by scanning the Deadswitch program's accounts, but they would only see wallet addresses and percentages — no personal information.

Q: What if one of my beneficiaries' wallets is compromised or lost? A: Update your vault with the new wallet address. If the beneficiary wallet is compromised before you update, the tokens would be sent to the compromised wallet. This is why we

recommend reviewing and updating your vault periodically.

Q: Is there a minimum amount to create a vault? A: No minimum asset value. The only cost is the Solana transaction fee (~~\$0.001~~) and rent for the vault account (0.02 SOL, refundable upon vault closure).

Q: What about taxes? A: Deadswitch does not provide tax advice. Crypto inheritance may be subject to estate taxes, income taxes, or capital gains taxes depending on your jurisdiction. Consult a tax professional.

Deadswitch — Because your crypto shouldn't die with you.

Copyright 2026 Deadswitch Labs. Released under MIT License.